UNIVERSIDADE DO VALE DO ITAJAÍ
CIÊNCIA DA COMPUTAÇÃO – PROJETO DE SISTEMAS DIGITAIS
PROF. PhD. DOUGLAS ROSSI DE MELO

GUSTAVO HENRIQUE STAHL MÜLLER

PROJETO DE CIRCUITOS COMBINACIONAIS - TMR

ITAJAÍ
2022

SUMÁRIO

1. INTRODUÇÃO

Esse trabalho tem como objetivo consolidar os conhecimentos de projeto de circuitos combinacionais e uma técnica de tolerância a falhas, envolvendo a criação dos três seguintes circuitos usando as ferramentas VHDL, EDA Playground e Quartus Prime:

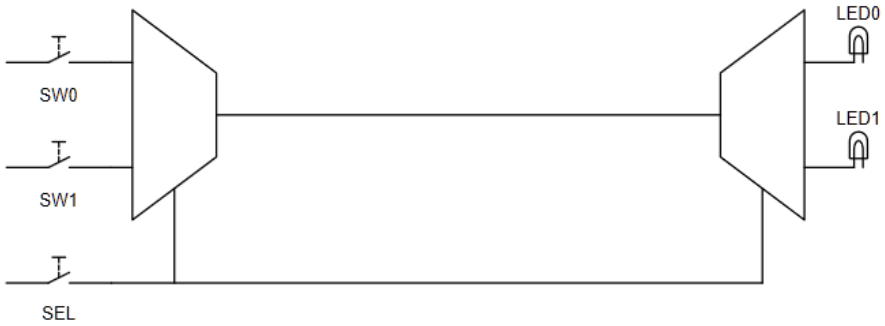
1. Um circuito de compartilhamento de canal.
2. Um circuito de compartilhamento de canal com uma entrada adicional para fins de simulação de falhas.
3. Um circuito de compartilhamento de canal que inclui um circuito TMR e três entradas adicionais para fins de simulação de falhas.

2. DESENVOLVIMENTO

2.1. CIRCUITO DE COMPARTILHAMENTO DE CANAL

Esse circuito tem como objetivo acionar um LED específico de saída com base no valor de um seletor. Para isso, foi usado o multiplexador 2-para-1 e o demultiplexador 1-para-2 criados no projeto passado.

Figura 1 – Diagrama abstrato do circuito de compartilhamento de canal.



Fonte: O autor.

Figura 2 – Tabela verdade do circuito de compartilhamento de canal.

SW(0)	SW(1)	SEL	LED(0)	LED(1)
0	0	0	0	0
0	0	1	0	0
0	1	0	0	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	1	0
1	1	1	0	1

Fonte: O autor.

Após definir a implementação do circuito é possível codificar a entidade, arquitetura e testbench:

Figura 3 – Entidade do circuito de compartilhamento de canal.

```
library IEEE;
use IEEE.std_logic_1164.all;

entity circuit_channel_sharing is
  port
  (
    i_SW: in std_logic_vector (1 downto 0);
    i_SEL: in std_logic;
```

```
    o_LED: out std_logic_vector (1 downto 0)
  );
end circuit_channel_sharing;
```

Fonte: O autor.

Figura 3 – Arquitetura do circuito de compartilhamento de canal.

```
architecture arch_circuit_channel_sharing of circuit_channel_sharing is
  component mux2_1bit is
    port
    (
      i_SEL : in std_logic;
      i_A : in std_logic;
      i_B : in std_logic;
      o_S : out std_logic
    );
  end component;

  component demux1_2bit is
    port
    (
      i_A: in std_logic;
      i_SEL: in std_logic;
      o_S: out std_logic_vector (1 downto 0)
    );
  end component;

  signal w_MUX: std_logic;

begin
  u_mux2_1bit: mux2_1bit port map (i_SEL, i_SW(0), i_SW(1), w_MUX);
  u_demux1_2bit: demux1_2bit port map (w_MUX, i_SEL, O_LED);

  process(i_SW, i_SEL)
  begin
  end process;
end arch_circuit_channel_sharing;
```

Fonte: O autor.

Figura 4 – Testbench do circuito de compartilhamento de canal.

```
-- Code your testbench here
library IEEE;
use IEEE.std_logic_1164.all;

entity circuit_channel_sharing_tb is
end circuit_channel_sharing_tb;

architecture arch_circuit_channel_sharing_tb of circuit_channel_sharing_tb is
  component circuit_channel_sharing is
    port
    (
      i_SW: in std_logic_vector (1 downto 0);
      i_SEL: in std_logic;
```

```
      o_LED: out std_logic_vector (1 downto 0)
    );
  end component;

  signal w_SW: std_logic_vector (1 downto 0);
  signal w_SEL: std_logic;
  signal w_LED: std_logic_vector (1 downto 0);

  begin
    u_design: circuit_channel_sharing port map (w_SW, w_SEL, w_LED);

    process
    begin
      w_SW(0) <= '0';
      w_SW(1) <= '0';
      w_SEL <= '0';
      wait for 1ns;

      assert (w_LED(0) = '0') report "Fail 0/0/0 -> output(0) == 1" severity error;
      assert (w_LED(1) = '0') report "Fail 0/0/0 -> output(1) == 1" severity error;

      w_SW(0) <= '0';
      w_SW(1) <= '0';
      w_SEL <= '1';
      wait for 1ns;

      assert (w_LED(0) = '0') report "Fail 0/0/1 -> output(0) == 1" severity error;
      assert (w_LED(1) = '0') report "Fail 0/0/1 -> output(1) == 1" severity error;

      w_SW(0) <= '0';
      w_SW(1) <= '1';
      w_SEL <= '0';
      wait for 1ns;

      assert (w_LED(0) = '0') report "Fail 0/1/0 -> output(0) == 1" severity error;
      assert (w_LED(1) = '0') report "Fail 0/1/0 -> output(1) == 1" severity error;

      w_SW(0) <= '0';
      w_SW(1) <= '1';
      w_SEL <= '1';
      wait for 1ns;

      assert (w_LED(0) = '0') report "Fail 0/1/1 -> output(0) == 1" severity error;
      assert (w_LED(1) = '1') report "Fail 0/1/1 -> output(1) == 1" severity error;

      w_SW(0) <= '1';
      w_SW(1) <= '0';
      w_SEL <= '0';
      wait for 1ns;

      assert (w_LED(0) = '1') report "Fail 1/0/0 -> output(0) == 1" severity error;
      assert (w_LED(1) = '0') report "Fail 1/0/0 -> output(1) == 1" severity error;

      w_SW(0) <= '1';
      w_SW(1) <= '0';
      w_SEL <= '1';
      wait for 1ns;
```

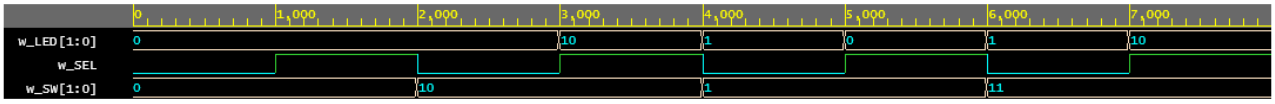
```
      assert (w_LED(0) = '0') report "Fail 1/0/1 -> output(0) == 1" severity error;
      assert (w_LED(1) = '0') report "Fail 1/0/1 -> output(1) == 1" severity error;

      w_SW(0) <= '1';
      w_SW(1) <= '1';
      w_SEL <= '0';
      wait for 1ns;

      assert (w_LED(0) = '1') report "Fail 1/1/0 -> output(0) == 1" severity error;
      assert (w_LED(1) = '0') report "Fail 1/1/0 -> output(1) == 1" severity error;

      w_SW(0) <= '1';
      w_SW(1) <= '1';
      w_SEL <= '1';
      wait for 1ns;

      assert (w_LED(0) = '0') report "Fail 1/1/1 -> output(0) == 1" severity error;
      assert (w_LED(1) = '1') report "Fail 1/1/1 -> output(1) == 1" severity error;

      wait;
    end process;
end arch_circuit_channel_sharing_tb;
```

Fonte: O autor.

Através do diagrama de forma de onda, é possível ver que o circuito se comporta corretamente:

- Quando SW(0) e SW(1) são 0, a saída dos dois LEDS são 0;
- Em todos os instantes em que SEL é 0 e SW(0) é 1, LED(0) é 1;
- Em todos os instantes em que SEL é 1 e SW(1) é 1, LED(1) é 1;

Figura 4 – Diagrama de forma de onda da simulação do circuito de compartilhamento de canal.



Fonte: O autor.

Todos os testes do testbench executam corretamente sem nenhuma mensagem de erro ou aviso:

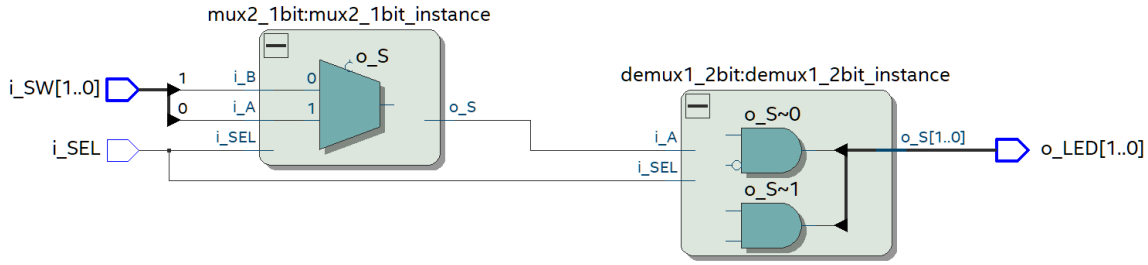
Figura 5 – Imagem do console após a execução do testbench do circuito de compartilhamento de canal.

```
# Aldec, Inc. Riviera-PRO version 2022.04.117.8517 built for Linux64 on May 04, 2022.
# HDL, SystemC, and Assertions simulator, debugger, and design environment.
# (c) 1999-2022 Aldec, Inc. All rights reserved.
# ELBREAD: Elaboration process.
# ELBREAD: Elaboration time 0.0 [s].
# KERNEL: Main thread initiated.
# KERNEL: Kernel process initialization phase.
# ELAB2: Elaboration final pass...
# ELAB2: Create instances ...
# KERNEL: Time resolution set to 1ps.
# ELAB2: Create instances complete.
# SLP: Started
# SLP: Elaboration phase ...
# SLP: Elaboration phase ... skipped, nothing to simulate in SLP mode : 0.0 [s]
# SLP: Finished : 0.0 [s]
# ELAB2: You do not have a license to run VHDL performance optimized simulation. Contact Aldec for ordering information - sales@aldec.com.
# ELAB2: Elaboration final pass complete - time: 0.0 [s].
# KERNEL: Warning: You are using the Riviera-PRO EDU Edition. The performance of simulation is reduced.
# KERNEL: Warning: Contact Aldec for available upgrade options - sales@aldec.com.
# KERNEL: Kernel process initialization done.
# Allocation: Simulator allocated 5401 kB (elbread=427 elab2=4831 kernel=142 sdf=0)
# KERNEL: ASDB file was created in location /home/runner/dataset.asdb
# KERNEL: PLI/VHPI kernel's engine initialization done.
# PLI: Loading library '/usr/share/Riviera-PRO/bin/libsystf.so'
# EXECUTION:: NOTE    : Test done.
# EXECUTION:: Time: 8 ns,  Iteration: 0,  Instance: /tb_mux2_1bit,  Process: line__28.
# KERNEL: Simulation has finished. There are no more test vectors to simulate.
# VSIM: Simulation has finished.
Finding VCD file...
./dump.vcd
[2022-09-17 21:20:11 EDT] Opening EPWave...
Done
```

Fonte: O autor.

Usando a ferramenta RTL Viewer do Quartus, é possível visualizar a implementação do circuito:

Figura 6 – Diagrama RTL do circuito de compartilhamento de canal.

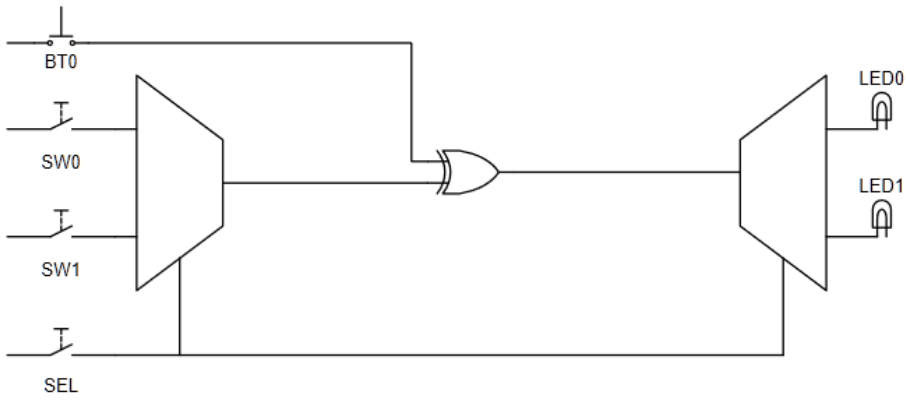


Fonte: O autor.

2.2. CIRCUITO DE COMPARTILHAMENTO DE CANAL COM SIMULAÇÃO DE FALHA

Esse circuito é composto do mesmo circuito passado adicionado de uma entrada que quando acionada inverte o valor de saída do multiplexador. A inversão ocorre através do uso de uma porta XOR, como a figura à seguir demonstra:

Figura 7 – Diagrama abstrato do circuito de compartilhamento de canal com simulação de falhas.



Fonte: O autor.

Figura 8 – Tabela verdade do circuito de compartilhamento de canal com simulação de falha.

SW(0)	SW(1)	SEL	BT	LED(0)	LED(1)
0	0	0	0	0	0
0	0	0	1	1	0
0	0	1	0	0	0
0	0	1	1	0	1
0	1	0	0	0	0
0	1	0	1	1	0
0	1	1	0	0	1
0	1	1	1	0	0
1	0	0	0	1	0
1	0	0	1	0	0
1	0	1	0	0	0
1	0	1	1	0	1
1	1	0	0	1	0
1	1	0	1	0	0
1	1	1	0	0	1
1	1	1	1	0	0

Fonte: O autor.

Figura 9 – Entidade do circuito de compartilhamento de canal com simulação de falha.

```
library IEEE;
use IEEE.std_logic_1164.all;
```

```
entity circuit_channel_sharing_with_fault is
  port
  (
    i_SW: in std_logic_vector (1 downto 0);
      i_SEL: in std_logic;
      i_BT: in std_logic;
      o_LED: out std_logic_vector (1 downto 0)
  );
end circuit_channel_sharing_with_fault;
```

Fonte: O autor.

Figura 10 – Arquitetura do circuito de compartilhamento de canal com simulação de falha.

```
architecture arch_circuit_channel_sharing_with_fault of circuit_channel_sharing_with_fault is
  component mux2_1bit is
    port
    (
        i_SEL : in std_logic;
        i_A : in std_logic;
        i_B : in std_logic;
        o_S : out std_logic
    );
  end component;

  component demux1_2bit is
    port
    (
        i_A: in std_logic;
        i_SEL: in std_logic;
        o_S: out std_logic_vector (1 downto 0)
    );
  end component;

  signal o_MUX: std_logic;

begin
  u_mux2_1bit: mux2_1bit port map (i_SEL, i_SW(0), i_SW(1), o_MUX);
  u_demux1_2bit: demux1_2bit port map (o_MUX XOR i_BT, i_SEL, O_LED);

  process(i_SW, i_SEL, i_BT)
  begin
  end process;
end arch_circuit_channel_sharing_with_fault;
```

Fonte: O autor.

Figura 11 – Testbench do circuito de compartilhamento de canal com simulação de falha.

```
-- Code your testbench here
library IEEE;
use IEEE.std_logic_1164.all;

entity circuit_channel_sharing_with_fault_tb is
end circuit_channel_sharing_with_fault_tb;

architecture arch_circuit_channel_sharing_with_fault_tb of circuit_channel_sharing_with_fault_tb is
  component circuit_channel_sharing_with_fault is
    port
    (
```

```
      i_SW: in std_logic_vector (1 downto 0);
      i_SEL: in std_logic;
      i_BT: in std_logic;
      o_LED: out std_logic_vector (1 downto 0)
    );
  end component;

  signal w_SW: std_logic_vector (1 downto 0);
  signal w_SEL: std_logic;
  signal w_BT: std_logic;
  signal w_LED: std_logic_vector (1 downto 0);

  begin
    u_design: circuit_channel_sharing_with_fault port map (w_SW, w_SEL, w_BT, w_LED);

    process
    begin
      w_SW(0) <= '0';
      w_SW(1) <= '0';
      w_SEL <= '0';
      w_BT <= '0';
      wait for 1ns;

      assert (w_LED(0) = '0') report "Fail 0/0/0/0 -> LED(0)" severity error;
      assert (w_LED(1) = '0') report "Fail 0/0/0/0 -> LED(1)" severity error;

      w_SW(0) <= '0';
      w_SW(1) <= '0';
      w_SEL <= '0';
      w_BT <= '1';
      wait for 1ns;

      assert (w_LED(0) = '1') report "Fail 0/0/0/1 -> LED(0)" severity error;
      assert (w_LED(1) = '0') report "Fail 0/0/0/1 -> LED(1)" severity error;

      w_SW(0) <= '0';
      w_SW(1) <= '0';
      w_SEL <= '1';
      w_BT <= '0';
      wait for 1ns;

      assert (w_LED(0) = '0') report "Fail 0/0/1/0 -> LED(0)" severity error;
      assert (w_LED(1) = '0') report "Fail 0/0/1/0 -> LED(1)" severity error;

      w_SW(0) <= '0';
      w_SW(1) <= '0';
      w_SEL <= '1';
      w_BT <= '1';
      wait for 1ns;

      assert (w_LED(0) = '0') report "Fail 0/0/1/1 -> LED(0)" severity error;
      assert (w_LED(1) = '1') report "Fail 0/0/1/1 -> LED(1)" severity error;

      w_SW(0) <= '0';
      w_SW(1) <= '1';
      w_SEL <= '0';
      w_BT <= '0';
      wait for 1ns;

      assert (w_LED(0) = '0') report "Fail 0/1/0/0 -> LED(0)" severity error;
      assert (w_LED(1) = '0') report "Fail 0/1/0/0 -> LED(1)" severity error;

      w_SW(0) <= '0';
      w_SW(1) <= '1';
```

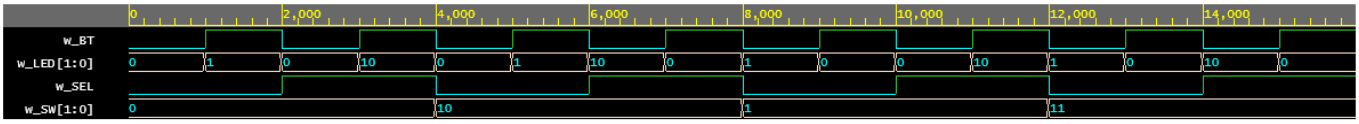
```
        w_SEL <= '0';
        w_BT <= '1';
        wait for 1ns;

        assert (w_LED(0) = '1') report "Fail 0/1/0/1 -> LED(0)" severity error;
        assert (w_LED(1) = '0') report "Fail 0/1/0/1 -> LED(1)" severity error;

        w_SW(0) <= '0';
        w_SW(1) <= '1';
        w_SEL <= '1';
        w_BT <= '0';
        wait for 1ns;

        assert (w_LED(0) = '0') report "Fail 0/1/1/0 -> LED(0)" severity error;
        assert (w_LED(1) = '1') report "Fail 0/1/1/0 -> LED(1)" severity error;

        w_SW(0) <= '0';
        w_SW(1) <= '1';
        w_SEL <= '1';
        w_BT <= '1';
        wait for 1ns;

        assert (w_LED(0) = '0') report "Fail 0/1/1/1 -> LED(0)" severity error;
        assert (w_LED(1) = '0') report "Fail 0/1/1/1 -> LED(1)" severity error;

        w_SW(0) <= '1';
        w_SW(1) <= '0';
        w_SEL <= '0';
        w_BT <= '0';
        wait for 1ns;

        assert (w_LED(0) = '1') report "Fail 1/0/0/0 -> LED(0)" severity error;
        assert (w_LED(1) = '0') report "Fail 1/0/0/0 -> LED(1)" severity error;

        w_SW(0) <= '1';
        w_SW(1) <= '0';
        w_SEL <= '0';
        w_BT <= '1';
        wait for 1ns;

        assert (w_LED(0) = '0') report "Fail 1/0/0/1 -> LED(0)" severity error;
        assert (w_LED(1) = '0') report "Fail 1/0/0/1 -> LED(1)" severity error;

        w_SW(0) <= '1';
        w_SW(1) <= '0';
        w_SEL <= '1';
        w_BT <= '0';
        wait for 1ns;

        assert (w_LED(0) = '0') report "Fail 1/0/1/0 -> LED(0)" severity error;
        assert (w_LED(1) = '0') report "Fail 1/0/1/0 -> LED(1)" severity error;

        w_SW(0) <= '1';
        w_SW(1) <= '0';
        w_SEL <= '1';
        w_BT <= '1';
        wait for 1ns;

        assert (w_LED(0) = '0') report "Fail 1/0/1/1 -> LED(0)" severity error;
        assert (w_LED(1) = '1') report "Fail 1/0/1/1 -> LED(1)" severity error;

        w_SW(0) <= '1';
        w_SW(1) <= '1';
        w_SEL <= '0';
```

```
      w_BT <= '0';
      wait for 1ns;

      assert (w_LED(0) = '1') report "Fail 1/1/0/0 -> LED(0)" severity error;
      assert (w_LED(1) = '0') report "Fail 1/1/0/0 -> LED(1)" severity error;

      w_SW(0) <= '1';
      w_SW(1) <= '1';
      w_SEL <= '0';
      w_BT <= '1';
      wait for 1ns;

      assert (w_LED(0) = '0') report "Fail 1/1/0/1 -> LED(0)" severity error;
      assert (w_LED(1) = '0') report "Fail 1/1/0/1 -> LED(1)" severity error;

      w_SW(0) <= '1';
      w_SW(1) <= '1';
      w_SEL <= '1';
      w_BT <= '0';
      wait for 1ns;

      assert (w_LED(0) = '0') report "Fail 1/1/1/0 -> LED(0)" severity error;
      assert (w_LED(1) = '1') report "Fail 1/1/1/0 -> LED(1)" severity error;

      w_SW(0) <= '1';
      w_SW(1) <= '1';
      w_SEL <= '1';
      w_BT <= '1';
      wait for 1ns;

      assert (w_LED(0) = '0') report "Fail 1/1/1/1 -> LED(0)" severity error;
      assert (w_LED(1) = '0') report "Fail 1/1/1/1 -> LED(1)" severity error;

      wait;
    end process;
end arch_circuit_channel_sharing_with_fault_tb;
```

Fonte: O autor.

Através do diagrama de forma de onda é possível comparar os resultados com a tabela verdade e identificar que o circuito gera as saídas corretas em todas as combinações possíveis de valores de entrada:

Figura 12 – Diagrama de forma de onda da simulação do circuito de compartilhamento de canal com simulação de falha.



Fonte: O autor.

Todos os testes do testbench executam corretamente sem nenhuma mensagem de erro ou aviso:

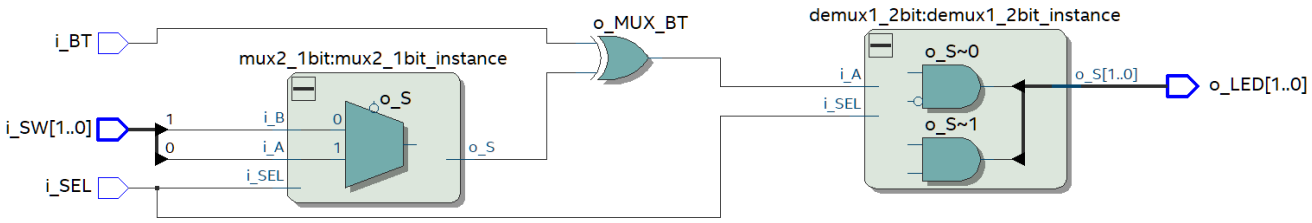
Figura 13 – Imagem do console após a execução do circuito de compartilhamento de canal com simulação de falha.

```
# Aldec, Inc. Riviera-PRO version 2022.04.117.8517 built for Linux64 on May 04, 2022.
# HDL, SystemC, and Assertions simulator, debugger, and design environment.
# (c) 1999-2022 Aldec, Inc. All rights reserved.
# ELBREAD: Elaboration process.
# ELBREAD: Elaboration time 0.0 [s].
# KERNEL: Main thread initiated.
# KERNEL: Kernel process initialization phase.
# ELAB2: Elaboration final pass...
# ELAB2: Create instances ...
# KERNEL: Time resolution set to 1ps.
# ELAB2: Create instances complete.
# SLP: Started
# SLP: Elaboration phase ...
# SLP: Elaboration phase ... skipped, nothing to simulate in SLP mode : 0.0 [s]
# SLP: Finished : 0.0 [s]
# ELAB2: You do not have a license to run VHDL performance optimized simulation. Contact Aldec for ordering information - sales@aldec.com.
# ELAB2: Elaboration final pass complete - time: 0.0 [s].
# KERNEL: Warning: You are using the Riviera-PRO EDU Edition. The performance of simulation is reduced.
# KERNEL: Warning: Contact Aldec for available upgrade options - sales@aldec.com.
# KERNEL: Kernel process initialization done.
# Allocation: Simulator allocated 6424 kB (elbread=427 elab2=5854 kernel=142 sdf=0)
# KERNEL: ASDB file was created in location /home/runner/dataset.asdb
# KERNEL: PLI/VHPI kernel's engine initialization done.
# PLI: Loading library '/usr/share/Riviera-PRO/bin/libsystf.so'
# KERNEL: Simulation has finished. There are no more test vectors to simulate.
# VSIM: Simulation has finished.
Finding VCD file...
./dump.vcd
[2022-09-17 21:18:03 EDT] Opening EPWave...
Done
```

Fonte: O autor.

Usando a ferramenta RTL Viewer do Quartus, é possível visualizar a implementação do circuito:

Figura 14 – Diagrama RTL do circuito de compartilhamento de canal com simulação de falha.

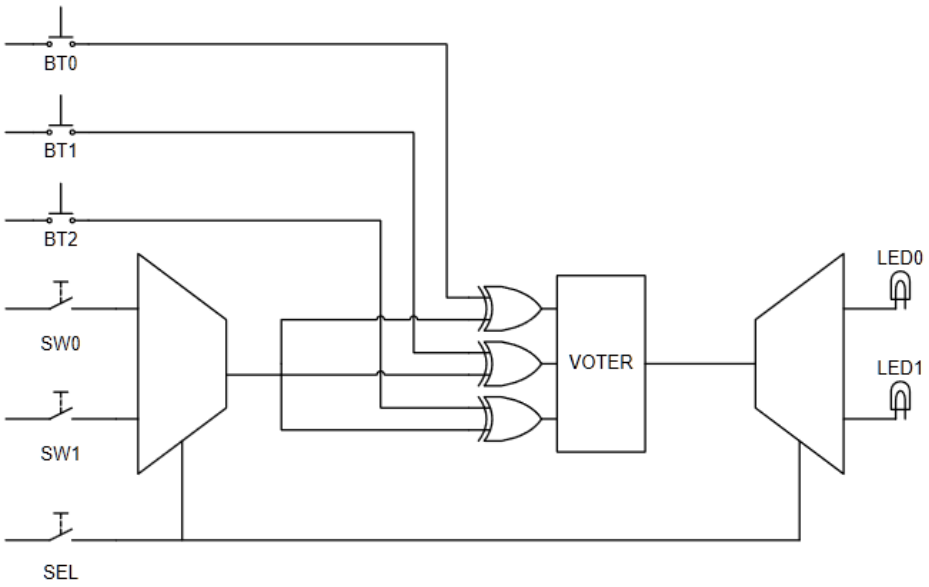


Fonte: O autor.

2.3. CIRCUITO DE COMPARTILHAMENTO DE CANAL COM TMR

Este circuito contém duas entradas adicionais para fins de simulação de falhas e um circuito TMR, que adiciona redundância à falhas na saída do multiplexador.

Figura 15 – Diagrama abstrato do circuito de compartilhamento de canal com TMR.



Fonte: O autor.

2.3.1 CIRCUITO TMR

O circuito TMR é um votador de maioria simples com três entradas e uma saída, onde a única saída do circuito será igual ao valor mais frequente das entradas.

Figura 16 – Tabela verdade do circuito TMR.

A	B	C	S
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Fonte: O autor.

Figura 17 – Equação booleana do circuito TMR.

$$S = BC + AC + AB + ABC;$$

Fonte: O autor.

Usando os teoremas de De Morgan, é possível simplificar a equação booleana acima para eliminar o termo “ABC”:

Figura 18 – Simplificação da equação booleana do circuito TMR.

$$S = BC + AC + AB + ABC;$$
$$\mathrm{S} = \mathrm{A(C} + \mathrm{BC} + \mathrm{B)} + \mathrm{BC};$$
$$\mathrm{S} = \mathrm{A(C(1} + \mathrm{B)} + \mathrm{B)} + \mathrm{BC};$$
$$\mathrm{S} = \mathrm{A(C} + \mathrm{B)} + \mathrm{BC};$$
$$\mathrm{S} = \mathrm{AC} + \mathrm{AB} + \mathrm{BC}$$

Fonte: O autor.

Após definir o comportamento esperado do circuito através da tabela verdade e a sua implementação através das equação booleana, é possível codificar a entidade, arquitetura e testbench:

Figura 19 – Entidade e arquitetura do circuito TMR.

```
library IEEE;
use IEEE.std_logic_1164.all;

entity tmr3_1bit is
  port
  (
    i_A: in std_logic;
    i_B: in std_logic;
    i_C: in std_logic;
    o_S: out std_logic
  );
end tmr3_1bit;

architecture arch_tmr3_1bit of tmr3_1bit is
begin
     o_S <= (i_A AND i_B) OR (i_A AND i_C) OR (i_B AND i_C);
end arch_tmr3_1bit;
```

Fonte: O autor.

Figura 20 – Entidade do circuito de compartilhamento de canal com TMR.

```
library IEEE;
use IEEE.std_logic_1164.all;
```

```
entity circuit_channel_sharing_with_tmr is
  port
  (
    i_SW: in std_logic_vector (1 downto 0);
      i_SEL: in std_logic;
      i_BT: in std_logic_vector (2 downto 0);
      o_LED: out std_logic_vector (1 downto 0)
  );
end circuit_channel_sharing_with_tmr;
```

Fonte: O autor.

Figura 21 – Arquitetura do circuito de compartilhamento de canal com TMR.

```
architecture arch_circuit_channel_sharing_with_tmr of circuit_channel_sharing_with_tmr is
  component mux2_1bit is
    port
    (
      i_SEL : in std_logic;
      i_A : in std_logic;
      i_B : in std_logic;
      o_S : out std_logic
     );
  end component;

  component demux1_2bit is
    port
    (
      i_A: in std_logic;
      i_SEL: in std_logic;
      o_S: out std_logic_vector (1 downto 0)
    );
  end component;

  component tmr3_1bit is
    port
    (
      i_A: in std_logic;
      i_B: in std_logic;
      i_C: in std_logic;
      o_S: out std_logic
    );
  end component;

  signal o_MUX: std_logic;
  signal o_TMR: std_logic;

begin
  mux2_1bit_instance: mux2_1bit port map (i_SEL, i_SW(0), i_SW(1), o_MUX);
  tmr3_1bit_instance: tmr3_1bit port map (o_MUX XOR i_BT(0), o_MUX XOR i_BT(1), o_MUX XOR
i_BT(2), o_TMR);
  demux1_2bit_instance: demux1_2bit port map (o_TMR, i_SEL, O_LED);

  process(i_SW, i_SEL, i_BT)
  begin
  end process;
```

```
end arch_circuit_channel_sharing_with_tmr;
```

Fonte: O autor.

Figura 22 – Testbench do circuito TMR.

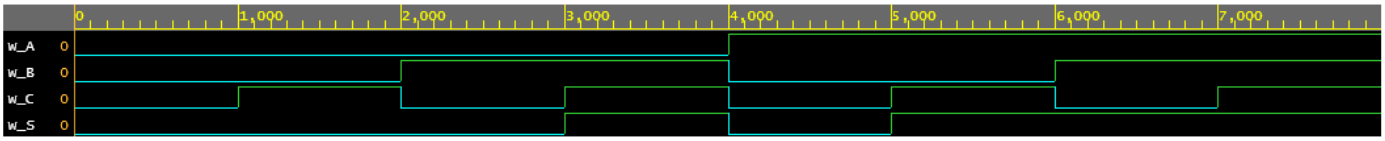
```
-- Code your testbench here
library IEEE;
use IEEE.std_logic_1164.all;

entity tmr3_1bit_tb is
end tmr3_1bit_tb;

architecture arch_tmr3_1bit_tb of tmr3_1bit_tb is
  component tmr3_1bit is
    port
    (
      i_A: in std_logic;
          i_B: in std_logic;
          i_C: in std_logic;
          o_S: out std_logic
    );
  end component;

  signal w_A: std_logic;
  signal w_B: std_logic;
  signal w_C: std_logic;
  signal w_S: std_logic;

  begin
    u_design: tmr3_1bit port map (w_A, w_B, w_C, w_S);

    process
    begin
        w_A <= '0';
        w_B <= '0';
        w_C <= '0';

        wait for 1ns;
        assert (w_S = '0') report "Fail 0/0/0 -> S" severity error;

        w_A <= '0';
        w_B <= '0';
        w_C <= '1';

        wait for 1ns;
        assert (w_S = '0') report "Fail 0/0/1 -> S" severity error;

        w_A <= '0';
        w_B <= '1';
        w_C <= '0';

        wait for 1ns;
        assert (w_S = '0') report "Fail 0/1/0 -> S" severity error;

        w_A <= '0';
        w_B <= '1';
        w_C <= '1';

        wait for 1ns;
        assert (w_S = '1') report "Fail 0/1/1 -> S" severity error;

        w_A <= '1';
        w_B <= '0';
        w_C <= '0';
```

```
        wait for 1ns;
        assert (w_S = '0') report "Fail 1/0/0 -> S" severity error;

        w_A <= '1';
        w_B <= '0';
        w_C <= '1';

        wait for 1ns;
        assert (w_S = '1') report "Fail 1/0/1 -> S" severity error;

        w_A <= '1';
        w_B <= '1';
        w_C <= '0';

        wait for 1ns;
        assert (w_S = '1') report "Fail 1/1/0 -> S" severity error;

        w_A <= '1';
        w_B <= '1';
        w_C <= '1';

        wait for 1ns;
        assert (w_S = '1') report "Fail 1/1/1 -> S" severity error;

        wait;
    end process;
end arch_tmr3_1bit_tb;
```

Fonte: O autor.

Através do diagrama de forma de onda é possível comparar os resultados com a tabela verdade e identificar que o circuito gera as saídas corretas em todas as combinações possíveis de valores de entrada:

Figura 23 – Diagrama de forma de onda da simulação do circuito TMR.



Fonte: O autor.

Todos os testes do testbench executam corretamente sem nenhuma mensagem de erro ou aviso:

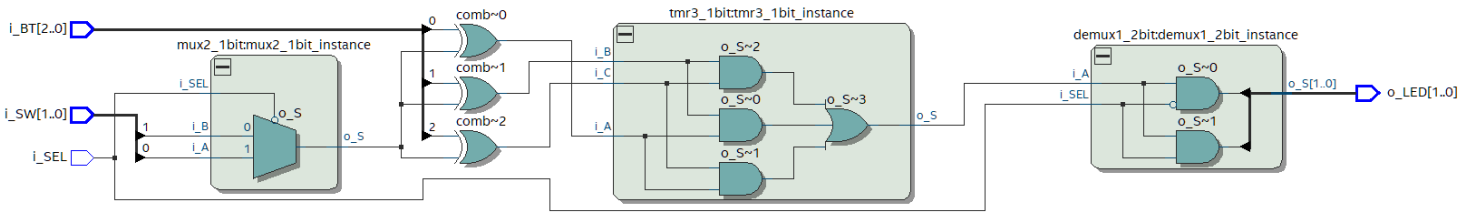
Figura 24 – Imagem do console após a execução do circuito de compartilhamento de canal com TMR.

```
# Aldec, Inc. Riviera-PRO version 2022.04.117.8517 built for Linux64 on May 04, 2022.
# HDL, SystemC, and Assertions simulator, debugger, and design environment.
# (c) 1999-2022 Aldec, Inc. All rights reserved.
# ELBREAD: Elaboration process.
# ELBREAD: Elaboration time 0.0 [s].
# KERNEL: Main thread initiated.
# KERNEL: Kernel process initialization phase.
# ELAB2: Elaboration final pass...
# ELAB2: Create instances ...
# KERNEL: Time resolution set to 1ps.
# ELAB2: Create instances complete.
# SLP: Started
# SLP: Elaboration phase ...
# SLP: Elaboration phase ... skipped, nothing to simulate in SLP mode : 0.0 [s]
# SLP: Finished : 0.0 [s]
# ELAB2: You do not have a license to run VHDL performance optimized simulation. Contact Aldec for ordering information - sales@aldec.com.
# ELAB2: Elaboration final pass complete - time: 0.0 [s].
# KERNEL: Warning: You are using the Riviera-PRO EDU Edition. The performance of simulation is reduced.
# KERNEL: Warning: Contact Aldec for available upgrade options - sales@aldec.com.
# KERNEL: Kernel process initialization done.
# Allocation: Simulator allocated 6424 kB (elbread=427 elab2=5854 kernel=142 sdf=0)
# KERNEL: ASDB file was created in location /home/runner/dataset.asdb
# KERNEL: PLI/VHPI kernel's engine initialization done.
# PLI: Loading library '/usr/share/Riviera-PRO/bin/libsystf.so'
# KERNEL: Simulation has finished. There are no more test vectors to simulate.
# VSIM: Simulation has finished.
Finding VCD file...
./dump.vcd
[2022-09-17 21:18:03 EDT] Opening EPWave...
Done
```

Fonte: O autor.

Usando a ferramenta RTL Viewer do Quartus, é possível visualizar a implementação do circuito:

Figura 25 – Diagrama RTL do circuito de compartilhamento de canal com TMR.



Fonte: O autor.